Name (LN, FN):	Quizon, Marc Lesley E.	Date:	**January 13, 2026**
Section:	S18	Grade:	

2.0 Unix / Linux Command Line Interface (CLI) Basics - File Management

This section of the course provides information how files and directories are organized in the Unix and Linux system. This section also shows how basic file management can done in the command line interface. The objectives of this section are:

- Be able to understand how file identifiers work,
- Be able to distinguish and use absolute and relative file names
- Be able to use basic file commands for file management

Unix file system, like other file systems of other operating systems, consists of *files* and each file has a *file name*. Unix / Linux file system consists of files and directories. A file is collection of related data while a directory is a special file that contain other files.

The Unix file system is organized as a hierarchy of directories where each directory can contain files or subdirectories. This hierarchy is called a *tree* where the *root* (/) is the top of the tree. Fig. 2.1 is the filesystem hierarchy of the Unix file system.

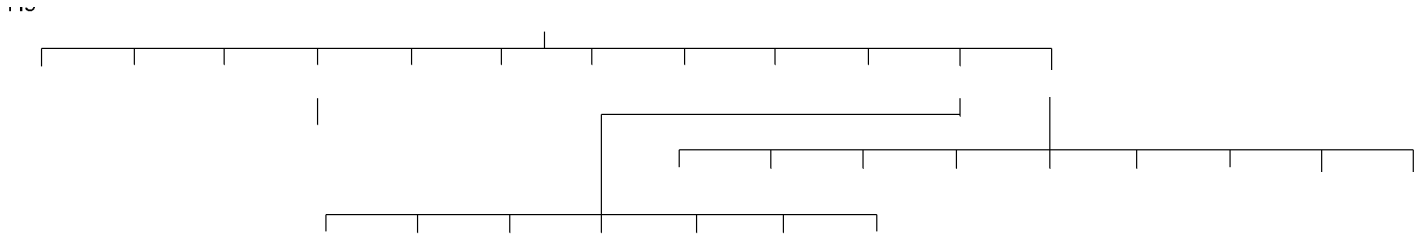

Fig. 2.1 Directories in the Linux file system

Directories in the Linux file hierarchy has a special description. Table 2.1 are the some of the description:

Table 2.1 Linux directories and description

Directory	Description
/	Root directory for the entire file system
/bin	Binaries for commands
/boot	Boot loader files
/dev	Device files
/etc	System configuration files
/home	Location for users' home directories
/lib	Library for binary files in `/bin` and `/sbin`

Depending on the Linux version or distribution, there may be additional directories aside from the mentioned above.

Each file in a directory has a *file identifier* (this is not synonymous to file name), which is a sequence of characters that uniquely identifies a file or directory. A file identifier can have at most 255 characters (this includes extension); any printable character can be used as to name a file but cannot contain "/". Although the characters "*", "?", ">", ">" or "|", it is not recommended because it has special meaning. Filenames in Unix are case sensitive.

File names in Unix, contains a sequence of file identifiers that designates a file. Slashes "/" are used to separate the sequence of file identifiers. There are two kinds of file name: absolute and relative file name. Absolute file name specifies the sequence of directories starting from the root (it actually starts with a "/") and can be used to refer to a file anywhere in the tree. Relative file name's starting point is the current directory or working directory. Relative file name can use symbols "." and ".." which represents the current directory and parent directory, respectively. Another symbol "~", represents the home directory. Home directory is the personal directory of a user where your files can be stored.

There are several basic file commands that can be used. Usual basic file commands needed are: to list files in a directory, change to another directory, create a new directory, move a file or directory, copy a file or directory, and remove a file or directory. Most of the commands can use absolute or relative filenames. The basic file commands are presented in Table 2.2.

Table 2.2 Basic file commands in the Unix / Linux operating system

Basic File Command	Description
ls	Lists files in a directory
cd	Change to another directory
mkdir	Create a new directory
mv	Move a file or directory
cp	Copy a file
rm or rmdir	Remove a file or directory
pwd	Shows your current directory

2.1. Traversing or Changing Directories

The "`cd`" command is used to change the working directory. Every time you login remotely, you will be in your home directory. Remember that a directory is a special file that contains files. This is the equivalent of "folders" in the Windows operating system.

Step 1 Login to Linux server using your assigned user account.

Step 2 While you are in the home directory, type the command "`pwd`".

2a. What was the message on the screen?

> The message was /home/lbysyads18user20

2b. What does the "`pwd`" command do?

> The pwd command tells the user the current directory he/she is in

Step 3 Type the command "`cd /etc`" and press enter.

3a. Did the prompt display differently? Write down the prompt that you see.

> The prompt changed to lbysyads18user20@SYSSVR1:/etc$. It changed because cd changes the directory. Hence, in our prompt the directory changed

3b. Type the command "`pwd`". What is your current working directory now?

> The current working directory now is /etc

Step 4 From the "`/etc`" directory, type the command "`cd /home/<username>`".

4a. Did you go to your "`home`" directory? How did you verify?

> Yes, it brought me back to my home directory. I verified by typing pwd and the prompt result was /home/lbysyads18user20

4b. Was the path used absolute or relative? How were you able to determine?

The path used was absolute, I determined this because our command starts with a /. This symbol represents the root directory, absolute file paths start from the root

4c. On the command prompt, type "`cd ../../etc`" and press enter. Were you able to go to the "`/etc`" directory? Was the path used absolute or relative? How were you able to determine?

Yes I am in the /etc directory. The path used is relative because we started from the current directory, not the root.

4d. From the "`/etc`" directory, type the command "`cd ~`". Were you able to go to the "`home`" directory? Which "`cd`" command you previously learned is this like? *Note: You can type the command "`pwd`" to list your current working directory.*

Yes I am back in the home directory. This command is similar to cd /home/lbysyads18user20 , they both bring me back to my home directory

2.2. Listing Directories

The "`ls`" command is used to list directories. There are a lot of options you can do with the command. It is able to display all files, directories only, and more options.

Step 5 On the command prompt, type the command "`cd /home/dir1`" and press enter.

5a. Enter the "`ls`" command. Describe the output you see.

I noticed that my prompt changed to lbysyads18user20@SYSSVR1:/home/dir1$. This shows that the directory has changed. I also see the ff:

dir1_1 file1 file12 file2 gile1 gile3 pile2
dir1_2 file11 file13 file3 gile2 pile1 pile3

These are the files that are in the dir1 directory.

5b. Enter the "`ls -A`" command. What is the option in this command? Describe the difference in the output from the previous question if there is any.

The output is:

.hidden_file dir1_2 file11 file13 file3 gile2 pile1 pile3
dir1_1 file1 file12 file2 gile1 gile3 pile2

The difference is that -A will show all files, even hidden ones, that are in the directory

5c. Enter the "`ls -a`" command. What is the option in this command? Describe the difference in the output from the previous questions (5a and 5b) if there is any.

The output is:

. .hidden_file dir1_2 file11 file13 file3 gile2 pile1 pile3
.. dir1_1 file1 file12 file2 gile1 gile3 pile2

The difference is the dots showing before .hidden_file and dir1_1. The (.) refers to the current directory, while the (..) refers to the parent directory. ls -A does not show these dots, but ls -a shows them

The "`ls`" command with option "`A`", shows all files in the directory including hidden files. Hidden files are files that start with ".". The "`a`" option has the same function as "`A`" option except that it also displays the "." (current) and ".." (parent) directories.

5d. Enter the "`ls -l`" command. What is the option in this command? Describe the difference in the output from the previous questions if there is any.

> The option is -l. This command is similar to ls, it does not show the hidden files. However, it shows the metadata of each file.

The first column on the "`ls -l`" shows the file permission of the file or directory (again, will be discussed later). A "`d`" as the first character shows which are the directories.

5e. Enter "`cd /etc`" and then the "`ls -d rc0.d`" command. What is the option in this command? Describe the difference in output when using the "`ls rc0.d`" command. *Note:* `rc0.d` *is a directory.*

> The option is -d. The difference with the two commands is that -d tells the system to output the directory. This can be used to check whether the directory exists or not. Whereas ls rc0.d simply lists the files that are in the rc0.d directory

The "`d`" displays if the "`rc0.d`" directory is there or not. While the "`ls`" command without the "`d`" option will display the contents of the directory "`rc0.d`".

5f. Enter the "`ls -R`" command. What is the option in this command? Describe the difference in output when using the "`ls`" command.

> The option is -R. This refers to recursive listing, its output will list the files in each folder and sub-folder until it has listed every file in each directory. Hence, the term 'recursive'.

The "`R`" option displays the directories recursively.

Step 6 Metacharacters are characters with special meaning in Unix. One of the metacharacters is "*" (asterisk).

6a. Go to the "`/home/dir1`" directory, type the command "`ls f*`". Describe the files that you see in the output.

> The output is "file1 file11 file12 file13 file2 file3". The reason these are shown is because f* is an option that means "show files that start with letter f".

6b. Enter the command "`ls p*`". Describe the files that you see in the output.

> The output is "pile1 pile2 pile3" The reason these are shown is because p* means "show files that start with p".

6c. Enter the command "`ls file?`" (question mark is included). Describe the files that you see in the output.

> The output is "file1 file2 file3". These are shown because 'file?' means "show files that contain 1 character after file". Hence file11, file12, and file13 are not shown.

6d. Enter the command "`ls file??`". Describe the files that you see in the output.

> The output is "file11 file12 file13". These are shown because 'file??' means "show files that contain 2 characters after file". Hence file1, file2, and file3 are not shown.

6e. What is the difference between the commands "`ls f*`" and "`ls file?`" and "`ls file??`"?

Firstly, * is a metacharacter that can represent zero or more of any character. While ? represents exactly one character.

ls f* – means “show files that start with f”. Since * can represent zero or more of any char, it won’t pay attention to what comes after f. It will simply output files that start with the characters stated before the * in the option .

ls file? – means “show files that start with ‘file’ and have exactly 1 character after.
ls file?? – means “show files that start with ‘file’ and have exactly 2 characters after.

6f. Enter the command “`ls [fp]ile1`”. Describe the files that you see in the output.

The output is “file1 pile1”. This option means “show files that start with f or p and has ‘ile1’ after”

6g. Enter the command “`ls [f-p]ile1`”. Describe the files that you see in the output.

The output is “file1 gile1 pile1”. This option means “show files that start with letters from f to p and has ‘ile1’ after”

6h. Enter the command “`ls [fp]ile?`”. Describe the files that you see in the output.

The output is “file1 file2 file3 pile1 pile2 pile3” . This option means “show files that start with letters f or p, and has ‘ile’ with exactly 1 character after”

6i. What is the difference between the commands “`ls [fp]ile1`” and “`ls [f-p]ile1`”?

The main difference is that [fp] indicates a file that starts with f or p. While [f-p] refers to files that start with letters from f to p.

The - indicates a range of characters. Without it, the system will just stick to the letters within the [ ].

The metacharacter “`*`” is a wildcard for multiple characters while “`?`” is wildcard for a single character. The metacharacter “`[..]`” is used to specify a range of characters.

2.3. Creating Directories

Directories in the Unix operating system can be created using the “`mkdir`” command. Naming directories has the same rules as naming files. Exercises on filenames will be tackled later.

Step 7 Go to your home directory by entering the command “`cd ~`”. Then create a directory “`coffees`” using the command “`mkdir coffees`”.

7a. Was the directory created? *Note: You can use the “`ls`” command to verify.*

Yes. When “ls coffees” was entered, no error message was shown. This means it exists.

7b. Enter the command “`cd coffees`” and press enter. Next, enter the command “`mkdir beans/robusta`”. Were you able to create the directory? Why or why not?

No, the directory was not created. We are not able to create the directory at the same time. An error message shows because the parent directory ‘beans’ does not exist at the time.

7c. Enter the command “`mkdir -p beans/robusta`”. Were you able to create the directory? What is the “`p`” option for?

The -p option refers to parent. This option allows the creation of an entire directory path. Hence, no error was encountered.

The "`p`" option allows creation of directories with its parent directory.

2.4. Files and Filenames

Files have attributes that can be viewed using the "`ls -l`" command. As shown in Fig. 2.2, the first column shows the permission of the file (which will be tackled later). Third column (marked in purple), shows the file ownership, fourth column (marked in red) shows the file size (in bytes), fifth column (marked in black) shows last modification date and last column (marked in yellow), shows the filename. Note that files with "`d`' in front of the file permission column indicates that the filename is a directory (Example: `drwxr-xr-x`).

Permission		File ownership		File size	Date last modified	Filename
-rw-r--r--	1	root	root	553122	Nov 5 2018	moduli
-rw-r--r--	1	root	root	1580	Nov 5 2018	ssh_config
-rw-------	1	root	root	1385	Jun 12 08:03	ssh_host_dsa_key
-rw-r--r--	1	root	root	603	Jun 12 08:03	ssh_host_dsa_key.pub
-rw-------	1	root	root	505	Jun 12 08:03	ssh_host_ecdsa_key
-rw-r--r--	1	root	root	175	Jun 12 08:03	ssh_host_ecdsa_key.pub
-rw-------	1	root	root	399	Jun 12 08:03	ssh_host_ed25519_key
-rw-r--r--	1	root	root	95	Jun 12 08:03	ssh_host_ed25519_key.pub
-rw-------	1	root	root	1823	Jun 12 08:03	ssh_host_rsa_key
-rw-r--r--	1	root	root	395	Jun 12 08:03	ssh_host_rsa_key.pub
-rw-r--r--	1	root	root	3264	Nov 5 2018	sshd_config

Fig. 2.2 File attributes as shown by the `ls` command

Fig. 2.2 File attributes as shown by the `ls` command

Empty files are files with a size of zero bytes. The "`touch`" command can be used to create empty files. The "`touch`" command will be used to practice creating files with different filenames.

In Unix, filenames can have up to 255 printable characters except for the "/" character. Although "*" and "?" can be used, these characters have special meaning.

Step 8 While in the directory "`coffees/beans/robusta`", enter the command "`touch robusta.txt`".

8a. Were you able to create the file? What is the size of the file? How were you able to determine the file size?

Yes, the file was successfully created. The file size is currently 0, I determined it by looking at the number before the date.

8b. Enter the command "`touch robusta/txt`". Were you able to create the file? Why or why not? What was the output?

No, the file was not created. We are already inside the robusta folder, this command tells the system to look for a robusta folder inside our robusta folder (which does not exist). Hence, it was not created. If we want a file named 'txt' inside our robusta folder, while in the robusta folder we just type 'touch txt'

8c. Enter the command "`touch robusta1`". Were you able to create the file? Why or why not?

Yes the file was successfully created, it is successful because it is the right command

8d. Enter the command "`mkdir robusta.doc`". Were you able to create the directory? Why or why not?

Yes the directory is created because the command is correct

8e. Enter the command "`ls`". Can you tell which one is a directory? What command should be used to be able to determine which one is a directory?

We are able to tell a file is a directory if it is blue in color. However, not all systems have color. Hence, we should use ls -l to see the file attributes. If the file permission column starts with a d, it is a directory.

8f. Enter the command "`ls robusta*`". Is there a difference from the earlier command? Can you tell which one is the directory? If yes, how?

This is the output

robusta.txt robusta1

robusta.doc:

Yes, the major difference is all the files are in white color. Moreover, this command filters files that start with robusta only. We can tell robusta.doc is the directory due to the semi-colon. Usually, directories have a : and its files are listed under it when we type ls -R

2.5. Copying Files

The `cp` command is used to copy files. The command can be used to copy one source to a destination or multiple sources to a directory.

Step 9 Enter the command "`cp robusta.txt robusta.html`".

9a. What happened after entering the command? Did you see the file "`robusta.html`"?

Yes, robusta.html exists now

9b. Enter the command "`cp robusta.txt robusta.doc`". What happened after entering the command? Did you see the file "`robusta.doc`"?

Nothing changed all the files are still there including robusta.doc

9c. Enter the command "`cd robusta.doc`" and then the command "`ls`". Did you see the file "`robusta.txt`"? Why is it there?

Yes robusta.txt is here. This is because of the command `cp robusta.txt robusta.doc ,` which simply means copy file robusta.txt into robusta.doc . Hence, in 9b we were not able to see the change but it has been applied and evident through 9c.

9d. Enter the command "`cd ..`" and then "`cp robusta.txt robusta.html`". Were you prompted if the file was to be overwritten?

No, there was no prompt if the file was to be overwritten

```
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta$ cd robusta.doc
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta/robusta.doc$ ls
robusta.txt
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta/robusta.doc$ cd ..
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta$ cp robusta.txt robusta.html
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta$ ls
robusta.doc  robusta.html  robusta.txt  robusta1
```

9e. Enter the command "`cp -i robusta.txt robusta.html`". Were you prompted if the file was to be overwritten? Why?

Yes, there is a prompt this time. This is because of the option -i which stands for interactive, its main purpose is to ask the user to overwrite existing files. This is good to keep data safe. Without -i, the file will be overwritten if it exists.

Answer "`y`" to the prompt.

2.6. Moving Files

In the Unix operating system, moving of files can be done by using the command "`mv`". The command can also be used to rename files.

Step 10 Enter the command "`mkdir -p ../arabica`" and press enter. Next, enter the command "`touch arabica.txt`". Use the command "`ls`" to verify the file is existent.

10a. Enter the command "`mv arabica.txt ../arabica`". Is the file "`arabica.txt`" still in your working directory? Where is it located now? What did the command "`mv`" do?

The file is no longer in the current directory, it has been moved to the arabica directory. The command 'mv' moves files.

10b. Enter the command "`touch arabica.txt`" under the directory "`<home directory>/coffees/beans/robusta`". Next, type the command "`mv arabica.txt ../arabica`". What happened? Were you prompted that you will be overwriting a file?

The file was moved and there was no prompt to overwrite the file.

10c. Enter the command "`touch arabica.txt`" under the directory "`<home directory>/coffees/beans/robusta`". Next, type the command "`mv -i arabica.txt ../arabica`". What happened? Were you prompted that you will be overwriting a file?

A prompt asking to overwrite the file was shown. This is because if -i.

Answer "`n`" to the prompt.

10d. Enter the command "`mv arabica.txt arabica.doc`". What happened? Is the file still there? Why or why not?

The directory lost arabica.txt and replaced it with arabica.doc. This happened because we tried to move a file to a directory that did not exist yet. So arabica.doc was created but arabica.txt could not be moved into it. Moreover, arabica.doc is not a directory due to the way it was created (It was not made using mkdir command)

10e. Explain the difference between the command "`cp arabica.txt ../arabica`" and "`mv arabica.txt ../arabica`".

The main difference is the command used. cp stands for copy while, mv stands for mv. When a user cp a file, it will still exist in the current directory and create a copy in the destination. However when a user mv a file, we are changing the location of the same file and it will not exist in the current directory anymore.

2.7. Removing Files

Removing or deleting files can be accomplished by using the command "`rm`". Be careful in using this command since deleted file are unrecoverable.

Step 11 Go to the "`<home directory>/coffees/beans/robusta`" directory. Type the command "`rm arabica.doc`".

11a. Was the file still there? Can it be found in another directory? Why or why not?

No, the file is now gone and deleted. It has been deleted because rm is the command to delete files

```
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta$ ls
arabica.doc  robusta.doc  robusta.html  robusta.txt  robusta1
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta$ rm arabica.doc
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta$ ls
robusta.doc  robusta.html  robusta.txt  robusta1
lbysyads18user20@SYSSVR1:~/coffees/beans/robusta$ cd ../
lbysyads18user20@SYSSVR1:~/coffees/beans$ ls
arabica  robusta
lbysyads18user20@SYSSVR1:~/coffees/beans$ cd arabica
lbysyads18user20@SYSSVR1:~/coffees/beans/arabica$ ls
arabica.txt
```

Step 12 Enter the command "`rm *`".

12a. What happened? Are there files or directories that were retained?

All other files were deleted except for robusta.doc. Whereas, robusta.html, robusta.txt, and robusta1 was deleted

12b. If a file or directory was retained, why was it retained?

A directory will be retained because it may still contain files. Similar to robusta.doc, it was retained because it still has robusta.txt in it.

Step 13 Go to the "`robusta.doc`" directory. Create the following directory "`bean_quality`" and "`bean_quality/perfect_cup`".

13a. Go to the "`<home directory>/coffees/beans/robusta`". Enter the command "`rm robusta.doc`". Were you able to remove the directory? Why or why not?

No, it cannot be removed because it is a directory. Moreover, there are numerous files inside.

13b. Enter the command "`rm -r robusta.doc`". Were you able to remove the directory? Why or why not? What is the option "`r`" for?

Yes the directory was deleted. -r is used to delete an entire directory with its contents. It will delete the files recursively until it is empty.

Step 14 Go to the ""`<home directory>/coffees/beans/arabica`" directory. Enter the command "`rm -i arabica.txt`".

14a. Was the file erased immediately? Is there a message before deletion?

No, there is a message. It states,
rm: remove regular empty file 'arabica.txt'?

14b. Answer "`y`" to the prompt. Was the file removed? What is the option "`i`" for?

Yes the file is now removed. For the rm command, -i is used to prompt the user to ask before delete

Step 15 Go to the "`<home directory>/coffees/beans`" directory. Enter the command "`rmdir arabica`".

15a. Was the directory erased immediately? Is there a message before deletion?

Yes, the directory was erased immediately. There was no message before deletion. The immediate deletion was successful because it was empty

The command "`rmdir`" is an alternative command for deleting directories. Note that the directory should be empty before you can delete it. If the directory is not empty, then use the "`rm -r <directory name>`" command. The command will delete both files and directories recursively.

2.8. Conclusion/Reflection

What are some of the differences in naming conventions of files and directories in Linux compared to another operating system that you are using (i.e., Windows/macOS)?

I primarily use macOS as my operating system. The first difference I notice is case-sensitivity, Linux is much more sensitive to case compared to macOS. I tested by doing "touch marc.txt", then "rm MARC.txt" on both systems. Linux prompted an error message saying the file did not exist, while macOS deleted the file. Moreover, the character : cannot be used for naming files in macOS. Whereas, in Linux it can be used. When I did "touch marc:stuff.txt" on both systems, macOS converted the : into a / , while Linux kept it as is.

Which command/s introduced in this activity did you find the most challenging to use? Why?

I found the commands easy to follow through. I like that the commands sound like their purpose. Such as mkdir – make directory, ls – list files, and cd – change directory. I found the options more challenging because it is quite ambiguous and the case of the single letter affects the output as well. Such as -r and -R. Moreover, these options can be used with multiple commands and lead to different outputs. I feel that the options is something I would need to pay more attention to.

Using a single command, how would you recursively display all the files in the directories in the `/etc` directory with their corresponding permissions (see sample screenshot)? Write the command on the given space below.

```
./X11:
total 8
drwxr-xr-x 2 root root 4096 Feb  2  2018 xkb
drwxr-xr-x 2 root root 4096 Mar  2 02:17 Xsession.d

./X11/xkb:
total 0

./X11/Xsession.d:
total 4
-rw-r--r-- 1 root root 880 Nov 30  2017 90gpg-agent

./xdg:
total 16
drwxr-xr-x 2 root root 4096 Mar  2 02:14 autostart
drwxr-xr-x 2 root root 4096 Aug  5  2019 systemd
-rw-r--r-- 1 root root  414 Mar 23  2018 user-dirs.conf
-rw-r--r-- 1 root root  418 Mar 23  2018 user-dirs.defaults

./xdg/autostart:
total 8
-rw-r--r-- 1 root root 228 Feb  2 08:21 snap-userd-autostart.desktop
-rw-r--r-- 1 root root 250 Mar 23  2018 xdg-user-dirs.desktop

./xdg/systemd:
total 0
lrwxrwxrwx 1 root root 18 Apr 20  2018 user -> ../../systemd/user
```

The command is ‘ls -lR /etc’. In this command we combine the option -l (for permissions) and -R (to recursively show each file), and use /etc to state the directory we want.